Relatório de evidências Sprint 2

Etapas do desafio

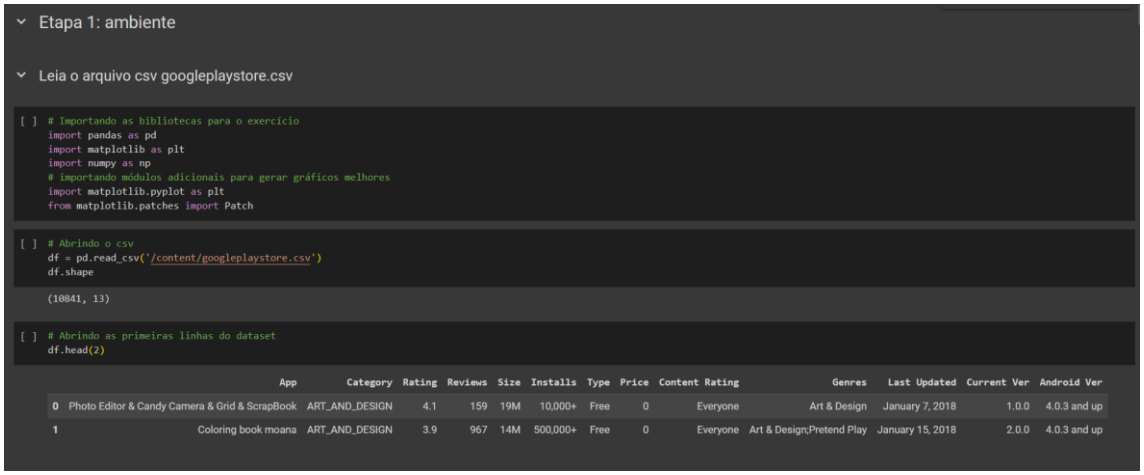
1- Instalação do Ambiente 2
2- Descoberta dos dados 3
3- Tratamento Complementar 4
4- Valores únicos 5
5- Valores ausentes 6
6- Valores duplicados 7
7- Gráfico top 5 8
8- Gráfico de pizza 9
9- App mais caro 11
10- Top 10 reviews 12
11 – Exercício 7 e 8 13

Figura 1 Etapa ambiente 2
Figura 2 Levantamento da orgamização do dataset 2
Figura 3 Descoberta dos dados 3
Figura 4 Descoberta dos dados cont. 3
Figura 5 Tratamento complementar 4
Figura 6 Descoberta dos dados cont. 4
Figura 7 Analisando valores únicos 5
Figura 8 Analisando valores únicos cont. 5
Figura 9 Analisando valores únicos cont. 5
Figura 10 Valores ausentes 6
Figura 11 Valores ausentes cont. 6
Figura 12 Valores ausentes cont. 6
Figura 13 Valores duplicados 7
Figura 14 Valores duplicados cont. 7
Figura 15 Valores duplicados cont. 7
Figura 16 Gráfico de barras - top 5 8
Figura 17 Gráfico de pizza 9
Figura 18 Gráfico de Pizza 2 10
Figura 19 App mais caro 11
Figura 20 Top 10 reviews 12
Figura 21 Lista com apps mais instalados e gráfico de pontos 13
Figura 22 Quantidade de App instalados em 2018 14
Figura 23 Aplicativo mais instalado em 2018 14
Figura 24 Gráfico com os aplicativos mais instalados por mês em 2018 14

1- Instalação do Ambiente

Começando pela criação do ambiente. Instalando as bibliotecas e verificando informações iniciais.

Figura 1 Etapa ambiente



Etapa 1: ambiente

Leia o arquivo csv googleplaystore.csv

```
[ ] # Importando as bibliotecas para o exercício
    import pandas as pd
    import matplotlib as plt
    import numpy as np
    # importando módulos adicionais para gerar gráficos melhores
    import matplotlib.pyplot as plt
    from matplotlib.patches import Patch
```

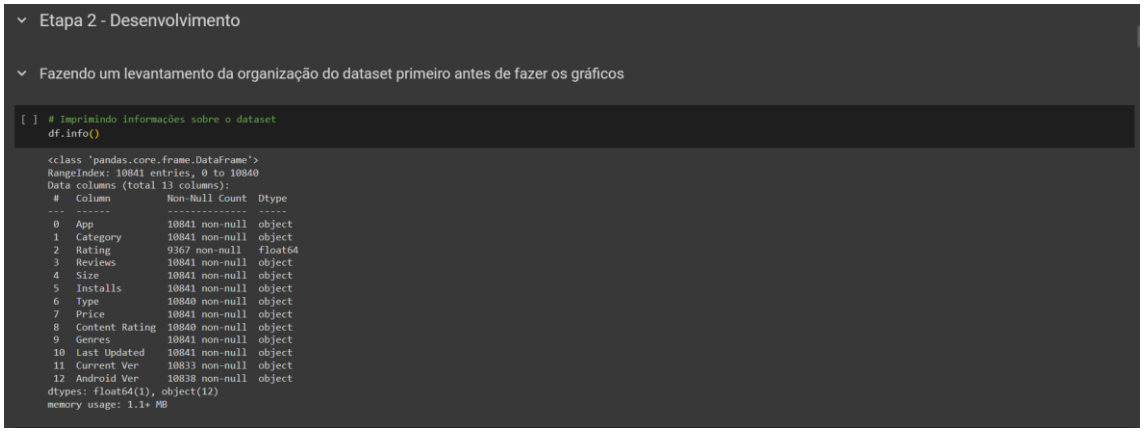
```
[ ] # Abrindo o csv
    df = pd.read_csv('/content/googleplaystore.csv')
    df.shape
```

```
(10841, 13)
```

```
[ ] # Abrindo as primeiras linhas do dataset
    df.head(2)
```

	App	Category	Rating	Reviews	Size	Installs	Type	Price	Content Rating	Genres	Last Updated	Current Ver	Android Ver
0	Photo Editor & Candy Camera & Grid & ScrapBook	ART_AND_DESIGN	4.1	159	19M	10,000+	Free	0	Everyone	Art & Design	January 7, 2018	1.0.0	4.0.3 and up
1	Coloring book moana	ART_AND_DESIGN	3.9	967	14M	500,000+	Free	0	Everyone	Art & Design;Pretend Play	January 15, 2018	2.0.0	4.0.3 and up

Figura 2 Levantamento da orgamização do dataset



Etapa 2 - Desenvolvimento

Fazendo um levantamento da organização do dataset primeiro antes de fazer os gráficos

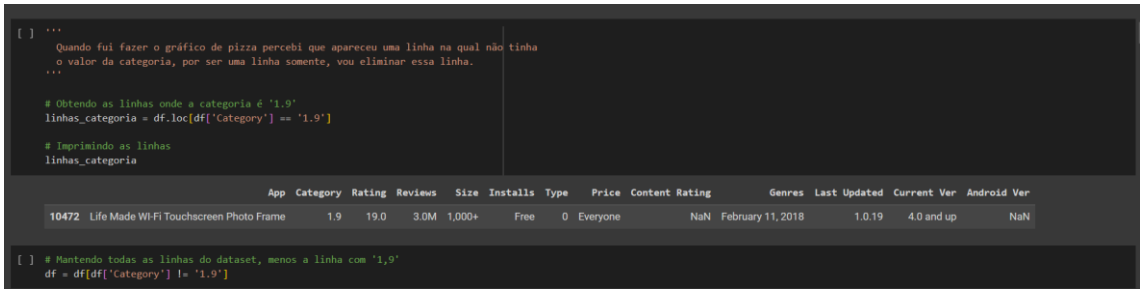
```
[ ] # Imprimindo informações sobre o dataset
    df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10841 entries, 0 to 10840
Data columns (total 13 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   App             10841 non-null  object
 1   Category        10841 non-null  object
 2   Rating          9367 non-null   float64
 3   Reviews         10841 non-null  object
 4   Size            10841 non-null  object
 5   Installs        10841 non-null  object
 6   Type            10840 non-null  object
 7   Price           10841 non-null  object
 8   Content Rating  10840 non-null  object
 9   Genres          10841 non-null  object
 10  Last Updated    10841 non-null  object
 11  Current Ver     10833 non-null  object
 12  Android Ver     10838 non-null  object
dtypes: float64(1), object(12)
memory usage: 1.1+ MB
```

2- Descoberta dos dados

A partir do primeiro momento de descoberta das informações, comecei a limpeza e tratamento básico das informações para criação de gráficos.

Figura 3 Descoberta dos dados



```
[ ] '''
      Quando fui fazer o gráfico de pizza percebi que apareceu uma linha na qual não tinha
      o valor da categoria, por ser uma linha somente, vou eliminar essa linha.
    '''

    # Obtendo as linhas onde a categoria é '1.9'
    linhas_categoria = df.loc[df['Category'] == '1.9']

    # Imprimindo as linhas
    linhas_categoria
```

	App	Category	Rating	Reviews	Size	Installs	Type	Price	Content Rating	Genres	Last Updated	Current Ver	Android Ver
10472	Life Made WI-Fi Touchscreen Photo Frame	1.9	19.0	3.0M	1,000+	Free	0	Everyone	NaN	February 11, 2018	1.0.19	4.0 and up	NaN

```
[ ] # Mantendo todas as linhas do dataset, menos a linha com '1,9'
    df = df[df['Category'] != '1.9']
```

Figura 4 Descoberta dos dados cont.



```
[ ] # Obtendo uma lista das categorias únicas
    categorias = df['Category'].unique()

    # Imprimindo a lista de categorias
    for categoria in categorias:
        print(categoria)
```

Alternar a visibilidade do cabeçalho

```
ART_AND_DESIGN
AUTO_AND_VEHICLES
BEAUTY
BOOKS_AND_REFERENCE
BUSINESS
COMICS
COMMUNICATION
DATING
EDUCATION
ENTERTAINMENT
EVENTS
FINANCE
FOOD_AND_DRINK
HEALTH_AND_FITNESS
HOUSE_AND_HOME
LIBRARIES_AND_DEMO
LIFESTYLE
GAME
FAMILY
MEDICAL
SOCIAL
SHOPPING
PHOTOGRAPHY
SPORTS
TRAVEL_AND_LOCAL
TOOLS
PERSONALIZATION
PRODUCTIVITY
PARENTING
WEATHER
VIDEO_PLAYERS
NEWS_AND_MAGAZINES
```

3- Tratamento Complementar

Como tratamento complementar converti os tipos dos dados para melhor construção dos gráficos.

Figura 5 Tratamento complementar



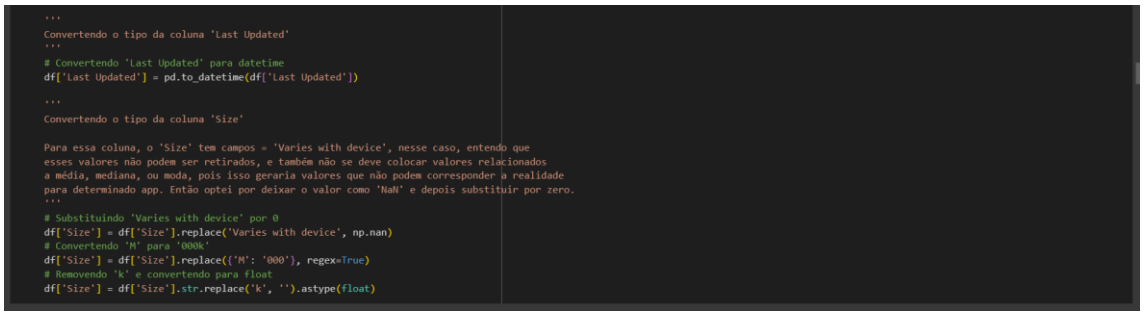
```
[ ] '''
    Convertendo o tipo da coluna 'Installs'
    '''
    # Convertendo 'Installs' para string
    df['Installs'] = df['Installs'].astype(str)
    # Removendo o '+' e as virgulas de 'Installs'
    df['Installs'] = df['Installs'].str.replace('+', '').str.replace(',', '')
    # Convertendo 'Installs' para int e 'Price' para float
    df['Installs'] = df['Installs'].astype(int)

    '''
    Convertendo o tipo da coluna 'Price'
    '''
    # Removendo o '$' de  'Price'
    df['Price'] = df['Price'].astype(str)
    # Removendo o '$' de 'Price'
    df['Price'] = df['Price'].str.replace('$', '')
    # Convertendo 'Price' para float
    df['Price'] = df['Price'].astype(float)

    '''
    Convertendo o tipo da coluna 'Reviews'
    '''
    # Convertendo 'Reviews' para int
    df['Reviews'] = df['Reviews'].astype(int)

    '''
    Convertendo o tipo da coluna 'Rating'
    '''
    # Convertendo 'Rating' para float
    df['Rating'] = df['Rating'].astype(float)
```

Figura 6 Descoberta dos dados cont.



```
    '''
    Convertendo o tipo da coluna 'Last Updated'
    '''
    # Convertendo 'Last Updated' para datetime
    df['Last Updated'] = pd.to_datetime(df['Last Updated'])

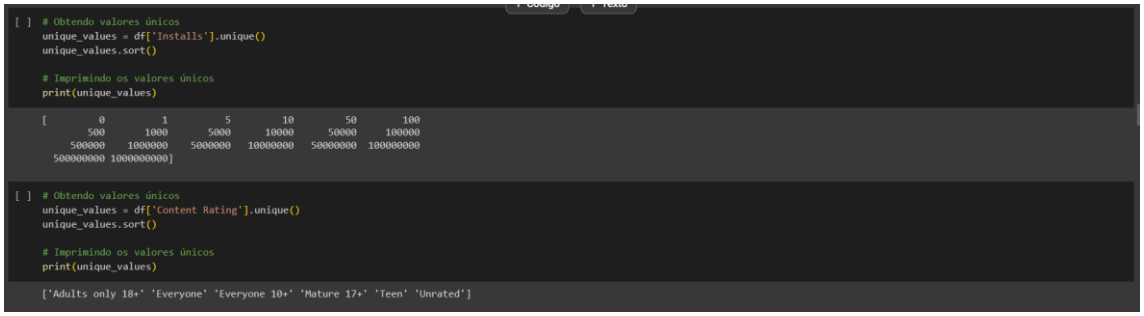
    '''
    Convertendo o tipo da coluna 'Size'

    Para essa coluna, o 'Size' tem campos = 'Varies with device', nesse caso, entendo que
    esses valores não podem ser retirados, e também não se deve colocar valores relacionados
    a média, mediana, ou moda, pois isso geraria valores que não podem corresponder a realidade
    para determinado app. Então optei por deixar o valor como 'NaN' e depois substituir por zero.
    '''
    # Substituindo 'Varies with device' por 0
    df['Size'] = df['Size'].replace('Varies with device', np.nan)
    # Convertendo 'M' para '000k'
    df['Size'] = df['Size'].replace({'M': '000'}, regex=True)
    # Removendo 'k' e convertendo para float
    df['Size'] = df['Size'].str.replace('k', '').astype(float)
```

4- Valores únicos

Verificando valores únicos para cada coluna para entender os resultados que obtive nos gráficos

Figura 7 Analisando valores únicos



```
[ ]  # Obtendo valores únicos
     unique_values = df['Installs'].unique()
     unique_values.sort()

     # Imprimindo os valores únicos
     print(unique_values)

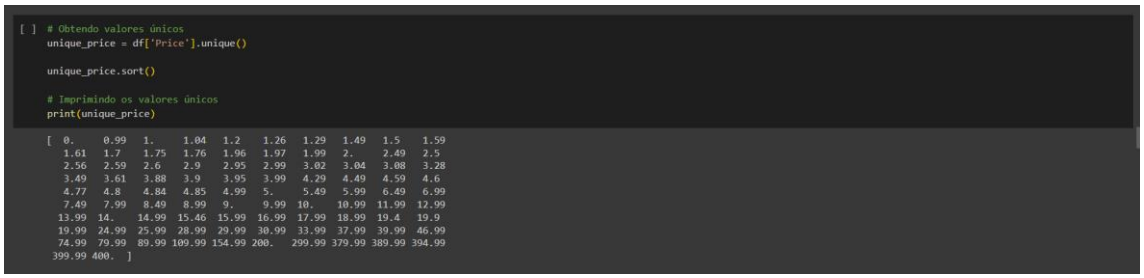
     [        0         1         5        10        50       100
            500      1000      5000     10000     50000    100000
         500000   1000000   5000000  10000000  50000000 100000000
       500000000 1000000000]

[ ]  # Obtendo valores únicos
     unique_values = df['Content Rating'].unique()
     unique_values.sort()

     # Imprimindo os valores únicos
     print(unique_values)

     ['Adults only 18+' 'Everyone' 'Everyone 10+' 'Mature 17+' 'Teen' 'Unrated']
```

Figura 8 Analisando valores únicos cont.



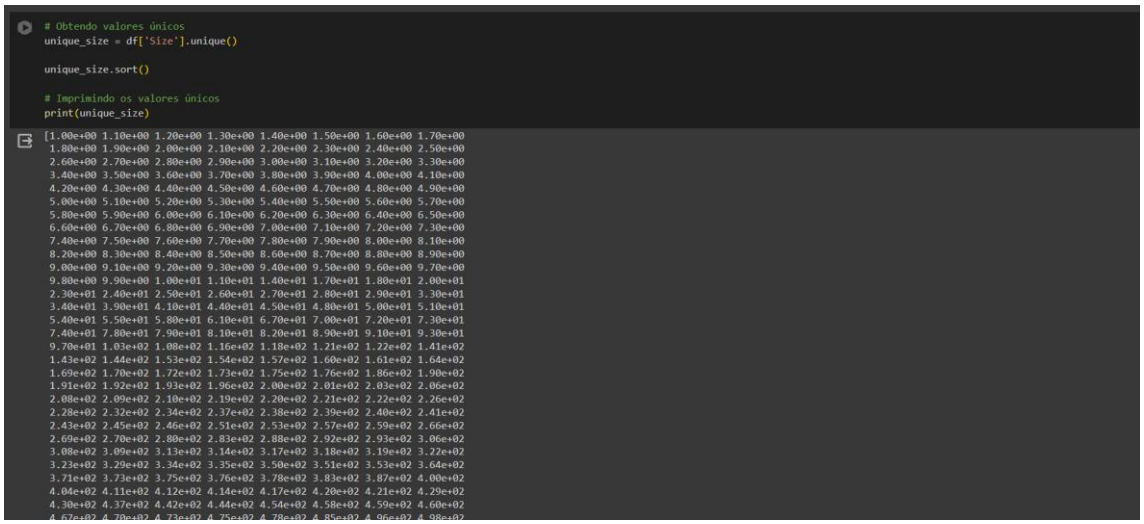
```
[ ]  # Obtendo valores únicos
     unique_price = df['Price'].unique()

     unique_price.sort()

     # Imprimindo os valores únicos
     print(unique_price)

     [  0.    0.99   1.    1.04   1.2    1.26   1.29   1.49   1.5    1.59
        1.61   1.7    1.75   1.76   1.96   1.97   1.99   2.     2.49   2.5
        2.56   2.59   2.6    2.9    2.95   2.99   3.02   3.04   3.08   3.28
        3.49   3.61   3.88   3.9    3.95   3.99   4.29   4.49   4.59   4.6
        4.77   4.8    4.84   4.85   4.99   5.     5.49   5.99   6.49   6.99
        7.49   7.99   8.49   8.99   9.     9.99  10.    10.99  11.99  12.99
       13.99  14.    14.99  15.46  15.99  16.99  17.99  18.99  19.4   19.9
       19.99  24.99  25.99  28.99  29.99  30.99  33.99  37.99  39.99  46.99
       74.99  79.99  89.99 109.99 154.99 200.   299.99 379.99 389.99 394.99
      399.99 400.  ]
```

Figura 9   Analisando valores únicos cont.



```
# Obtendo valores únicos
unique_size = df['Size'].unique()

unique_size.sort()

# Imprimindo os valores únicos
print(unique_size)

[1.00e+00 1.10e+00 1.20e+00 1.30e+00 1.40e+00 1.50e+00 1.60e+00 1.70e+00
 1.80e+00 1.90e+00 2.00e+00 2.10e+00 2.20e+00 2.30e+00 2.40e+00 2.50e+00
 2.60e+00 2.70e+00 2.80e+00 2.90e+00 3.00e+00 3.10e+00 3.20e+00 3.30e+00
 3.40e+00 3.50e+00 3.60e+00 3.70e+00 3.80e+00 3.90e+00 4.00e+00 4.10e+00
 4.20e+00 4.30e+00 4.40e+00 4.50e+00 4.60e+00 4.70e+00 4.80e+00 4.90e+00
 5.00e+00 5.10e+00 5.20e+00 5.30e+00 5.40e+00 5.50e+00 5.60e+00 5.70e+00
 5.80e+00 5.90e+00 6.00e+00 6.10e+00 6.20e+00 6.30e+00 6.40e+00 6.50e+00
 6.60e+00 6.70e+00 6.80e+00 6.90e+00 7.00e+00 7.10e+00 7.20e+00 7.30e+00
 7.40e+00 7.50e+00 7.60e+00 7.70e+00 7.80e+00 7.90e+00 8.00e+00 8.10e+00
 8.20e+00 8.30e+00 8.40e+00 8.50e+00 8.60e+00 8.70e+00 8.80e+00 8.90e+00
 9.00e+00 9.10e+00 9.20e+00 9.30e+00 9.40e+00 9.50e+00 9.60e+00 9.70e+00
 9.80e+00 9.90e+00 1.00e+01 1.10e+01 1.40e+01 1.70e+01 1.80e+01 2.00e+01
 2.30e+01 2.40e+01 2.50e+01 2.60e+01 2.70e+01 2.80e+01 2.90e+01 3.30e+01
 3.40e+01 3.90e+01 4.10e+01 4.40e+01 4.50e+01 4.80e+01 5.00e+01 5.10e+01
 5.40e+01 5.50e+01 5.80e+01 6.10e+01 6.70e+01 7.00e+01 7.20e+01 7.30e+01
 7.40e+01 7.80e+01 7.90e+01 8.10e+01 8.20e+01 8.90e+01 9.10e+01 9.30e+01
 9.70e+01 1.03e+02 1.08e+02 1.16e+02 1.18e+02 1.21e+02 1.22e+02 1.41e+02
 1.43e+02 1.44e+02 1.53e+02 1.54e+02 1.57e+02 1.60e+02 1.61e+02 1.64e+02
 1.69e+02 1.70e+02 1.72e+02 1.73e+02 1.75e+02 1.76e+02 1.86e+02 1.90e+02
 1.91e+02 1.92e+02 1.93e+02 1.96e+02 2.00e+02 2.01e+02 2.03e+02 2.06e+02
 2.08e+02 2.09e+02 2.10e+02 2.19e+02 2.20e+02 2.21e+02 2.22e+02 2.26e+02
 2.28e+02 2.32e+02 2.34e+02 2.37e+02 2.38e+02 2.39e+02 2.40e+02 2.41e+02
 2.43e+02 2.45e+02 2.46e+02 2.51e+02 2.53e+02 2.57e+02 2.59e+02 2.66e+02
 2.69e+02 2.70e+02 2.80e+02 2.83e+02 2.88e+02 2.92e+02 2.93e+02 3.06e+02
 3.08e+02 3.09e+02 3.13e+02 3.14e+02 3.17e+02 3.18e+02 3.19e+02 3.22e+02
 3.23e+02 3.29e+02 3.34e+02 3.35e+02 3.50e+02 3.51e+02 3.53e+02 3.64e+02
 3.71e+02 3.73e+02 3.75e+02 3.76e+02 3.78e+02 3.83e+02 3.87e+02 4.00e+02
 4.04e+02 4.11e+02 4.12e+02 4.14e+02 4.17e+02 4.20e+02 4.21e+02 4.29e+02
 4.30e+02 4.37e+02 4.42e+02 4.44e+02 4.54e+02 4.58e+02 4.59e+02 4.60e+02
 4.67e+02 4.70e+02 4.73e+02 4.75e+02 4.78e+02 4.85e+02 4.96e+02 4.98e+02
```

5- Valores ausentes

Verificando e tratando valores ausentes

Figura 10 Valores ausentes



```
# Contando valores NaN da coluna 'Size'
nan_count = df['Size'].isna().sum()
print(f"Quantidade de valores NaN: {nan_count}")

# Contando valores 0 da coluna 'Size'
zero_count = (df['Size'] == 0).sum()
print(f"Quantidade de valores 0: {zero_count}")
```

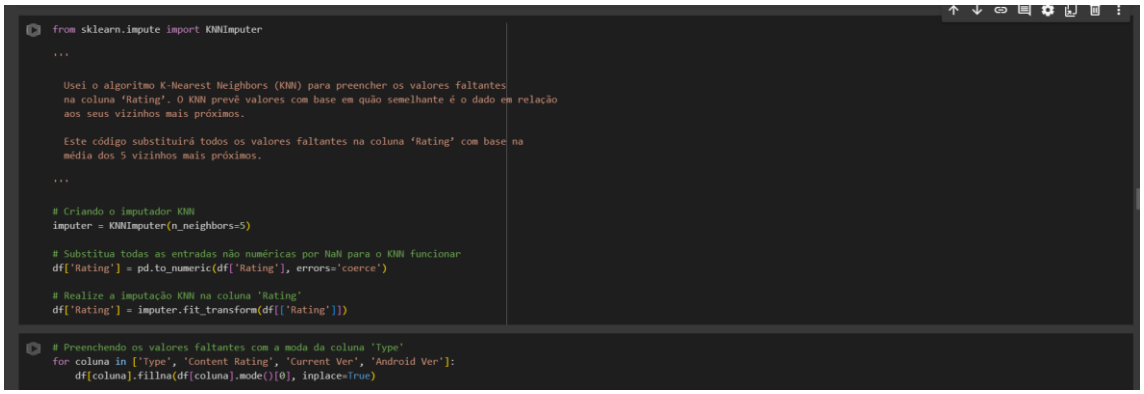
```
Quantidade de valores NaN: 1695
Quantidade de valores 0: 0
```

```
# Preenchendo os valores NaN da coluna 'Size' por 0
df['Size'] = df['Size'].fillna(0)
```

```
# faz a contagem dos valores nulos
df_na_sum = df.isna().sum()
df_na_sum
```

```
App                0
Category           0
Rating          1474
Reviews            0
Size               0
Installs           0
Type               1
Price              0
Content Rating     0
Genres             0
Last Updated       0
Current Ver        8
Android Ver        2
dtype: int64
```

Figura 11 Valores ausentes cont.



```
from sklearn.impute import KNNImputer

'''

  Usei o algoritmo K-Nearest Neighbors (KNN) para preencher os valores faltantes
  na coluna 'Rating'. O KNN prevê valores com base em quão semelhante é o dado em relação
  aos seus vizinhos mais próximos.

  Este código substituirá todos os valores faltantes na coluna 'Rating' com base na
  média dos 5 vizinhos mais próximos.

'''

# Criando o imputador KNN
imputer = KNNImputer(n_neighbors=5)

# Substitua todas as entradas não numéricas por NaN para o KNN funcionar
df['Rating'] = pd.to_numeric(df['Rating'], errors='coerce')

# Realize a imputação KNN na coluna 'Rating'
df['Rating'] = imputer.fit_transform(df[['Rating']])
```

```
# Preenchendo os valores faltantes com a moda da coluna 'Type'
for coluna in ['Type', 'Content Rating', 'Current Ver', 'Android Ver']:
    df[coluna].fillna(df[coluna].mode()[0], inplace=True)
```

Figura 12 Valores ausentes cont.



```
# faz a contagem dos valores nulos
df_na_sum = df.isna().sum()
df_na_sum
```

```
App                0
Category           0
Rating             0
Reviews            0
Size               0
Installs           0
Type               0
Price              0
Content Rating     0
Genres             0
Last Updated       0
Current Ver        0
Android Ver        0
dtype: int64
```

6- Valores duplicados

Nessa etapa verifiquei se há valores duplicados. Percebi que além de duplicados, havia linhas em que o nome do aplicativo estava repetido, porém havia atualizações nos reviews. Escolhi manter somente a linha com maior número de reviews por aplicativo. Por fim imprimi mais uma vez os dados do df para fazer uma verificação.

Figura 13 Valores duplicados

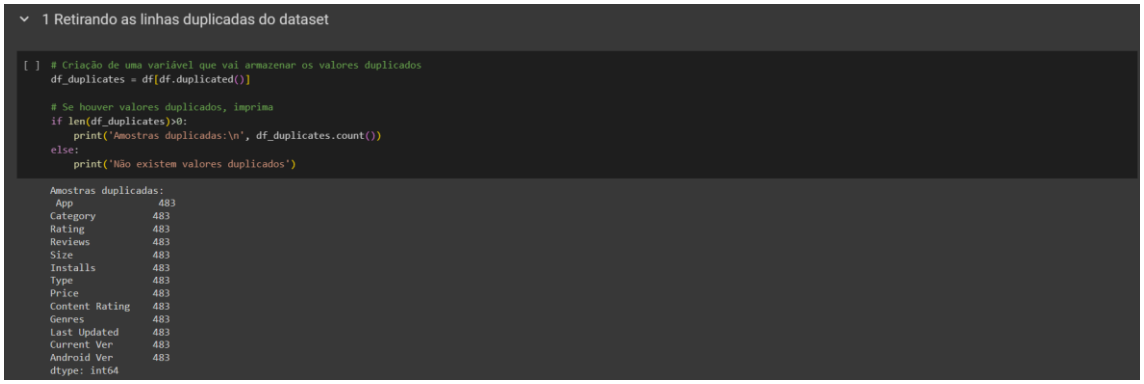


Figura 14 Valores duplicados cont.



Figura 15 Valores duplicados cont.



7- Gráfico top 5

Aqui comecei a plotar os gráficos pedidos no desafio. Primeiro o gráfico mostrando os top 5 aplicativos por instalação.

Figura 16 Gráfico de barras - top 5

```
2 Faça um gráfico de barras contendo os top 5 por números de instalação

'''
Nesse gráfico, notei que alguns apps apareciam mais de uma vez na lista, mesmo retirando os
valores duplicados. O que muda de uma linha para outra ou é o números de reviews ou a data de
atualização.
'''

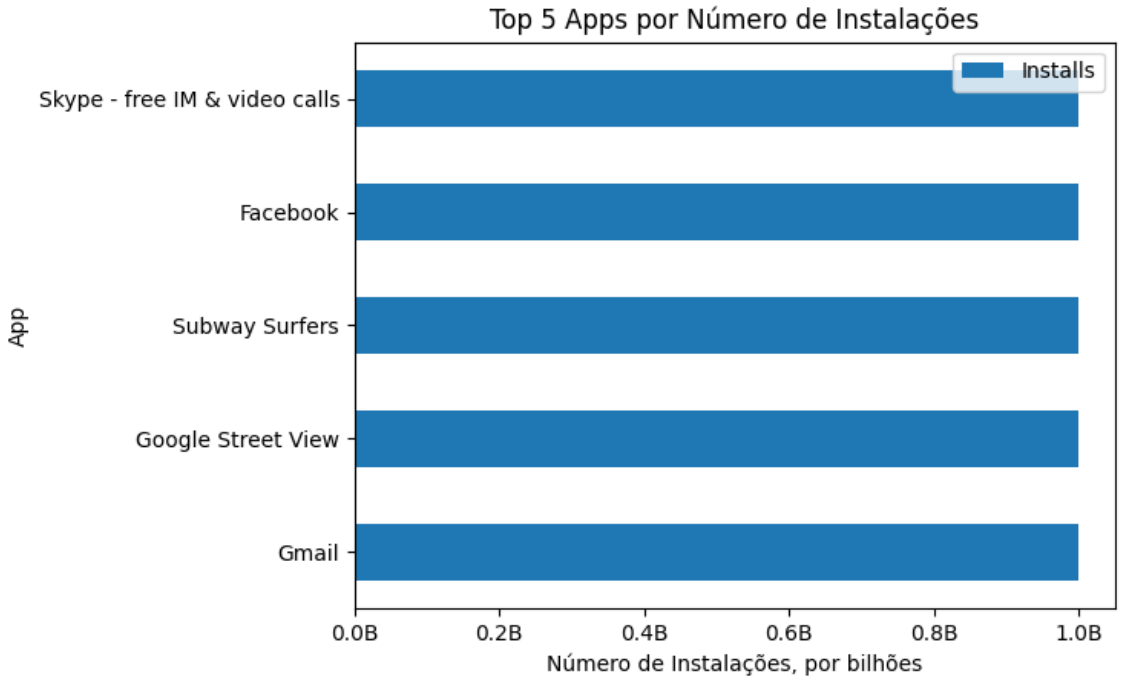
# Ordenando em ordem decrescente e pegando os top 5
top_5_apps = df.sort_values(by='Installs', ascending=False).head(5)

# Plotando o gráfico
ax = top_5_apps.plot(kind='barh', x='App', y='Installs')

# Ajustando os valores do eixo x para bilhões
ax.xaxis.set_major_formatter(lambda x, _: '{:.1f}B'.format(x*1e-9))

# Adicionando título ao gráfico e aos eixos
plt.title('Top 5 Apps por Número de Instalações')
plt.xlabel('Número de Instalações, por bilhões')
plt.ylabel('App')

# Mostrando o gráfico
plt.show()
```



8- Gráfico de pizza

Nesse ponto plotei o gráfico de pizza. Como havia muitas categorias, no primeiro, peguei as últimas categorias e uni em uma categoria chamada ‘outros’. Mas achei que talvez assim não iria corresponder ao escopo do que foi pedido, então plotei outro gráfico com todas, porém ao lado uma legenda com os nomes dos aplicativos.

Figura 17 Gráfico de pizza

```
# Definindo o número de categorias mostradas separadamente
num_categorias = 19

# Calcula a contagem e a porcentagem de cada categoria
category_counts = df['Category'].value_counts()
category_percentages = category_counts / category_counts.sum() * 100

# Agrupa as categorias com menos frequência em 'Outros'
if len(category_counts) > num_categorias:
    category_counts = category_counts[:num_categorias]
    category_counts.loc['Outros'] = df['Category'].value_counts()[num_categorias:].sum()

    category_percentages = category_percentages[:num_categorias]
    category_percentages.loc['Outros'] = df['Category'].value_counts(normalize=True)[num_categorias:].sum() * 100

# Cria a figura e os eixos
fig, ax = plt.subplots(figsize=(10, 7))

# Cria o gráfico de pizza
colors = plt.cm.tab20.colors[:len(category_counts.index)]
wedges, texts, autotexts = ax.pie(category_counts, labels=None, autopct="%1.1f%%", colors=colors, pctdistance=1.15)

# Cria a tabela de legenda
legend_labels = ['{}'.format(i) for i, in zip(category_counts.index)]

# Cria patches para a legenda -- serve para criar uma legenda personalizada para o gráfico--
patches = [Patch(facecolor=c, edgecolor=c, label=l) for c, l in zip(colors, legend_labels)]

# Adiciona a legenda ao gráfico
ax.legend(handles=patches, title="Categorias", loc="center left", bbox_to_anchor=(1.40, 0.5), borderaxespad=0.)

plt.title("Frequência das Categorias de Aplicativos", loc="center", fontsize=15)
plt.show()
```

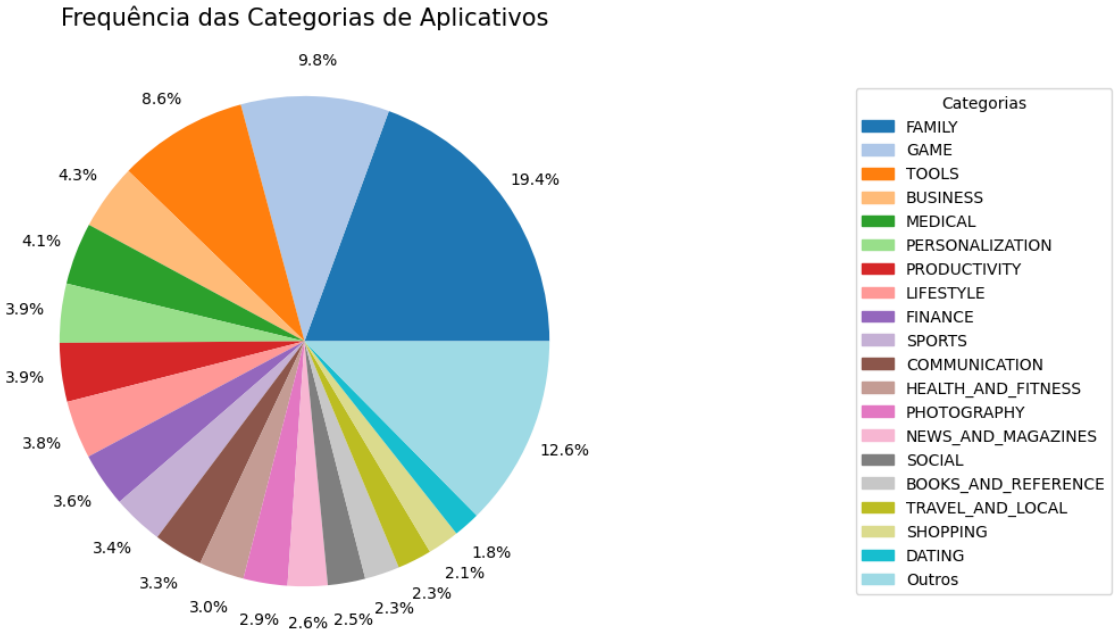


Figura 18 Gráfico de Pizza 2

```
[ ] '''
    Para poder cumprir os requisitos fiz outro gráfico com todas as colunas.
    '''

    # Calcula a contagem e a porcentagem de cada categoria
    category_counts = df['Category'].value_counts()
    category_percentages = category_counts / category_counts.sum() * 100

    # Cria a figura e os eixos
    fig, ax = plt.subplots(figsize=(13, 10))

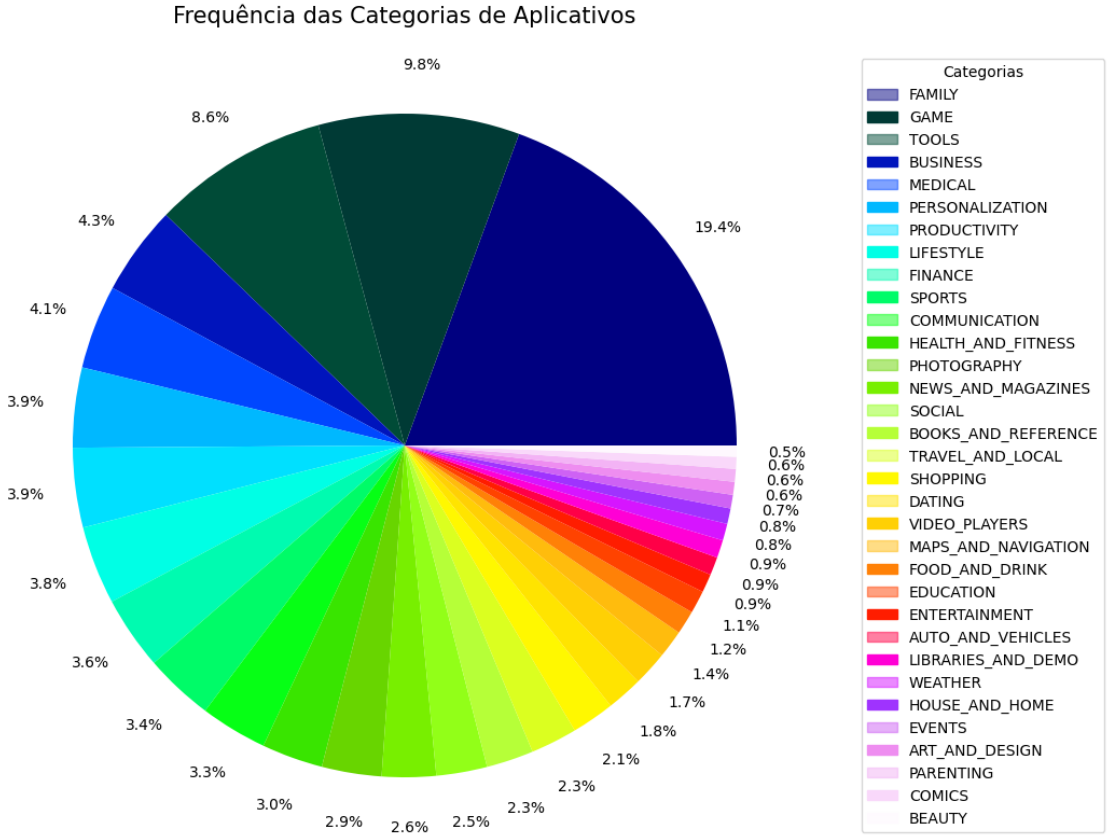
    # Cria o gráfico de pizza
    colors = plt.cm.gist_ncar(np.linspace(0, 1, len(category_counts.index)))
    wedges, texts, autotexts = ax.pie(category_counts, labels=None, colors=colors, autopct='%1.1f%%', pctdistance=1.15)

    # Cria a tabela de legenda
    legend_labels = ['{}'.format(i) for i, in zip(category_counts.index)]

    # Cria patches para a legenda -- serve para criar uma legenda personalizada para o gráfico--
    patches = [Patch(facecolor=c, edgecolor=c, label=l, alpha=0.5 if i%2==0 else 1) for i, (c, l) in enumerate(zip(colors, legend_labels))]

    # Adiciona a legenda ao gráfico
    ax.legend(handles=patches, title="Categorias", loc="center left", bbox_to_anchor=(1.05, 0.5), borderaxespad=0.)

    plt.title("Frequência das Categorias de Aplicativos", loc="center", fontsize=15)
    plt.show()
```



9- App mais caro

Continuei com o exercício seguinte imprimindo o app mais caro do dataset e também quantos apps são considerados ‘Mature 17+’.

Figura 19 App mais caro



10- Top 10 reviews

Seguindo, imprimi uma lista com 10 apps com maior número de reviews.

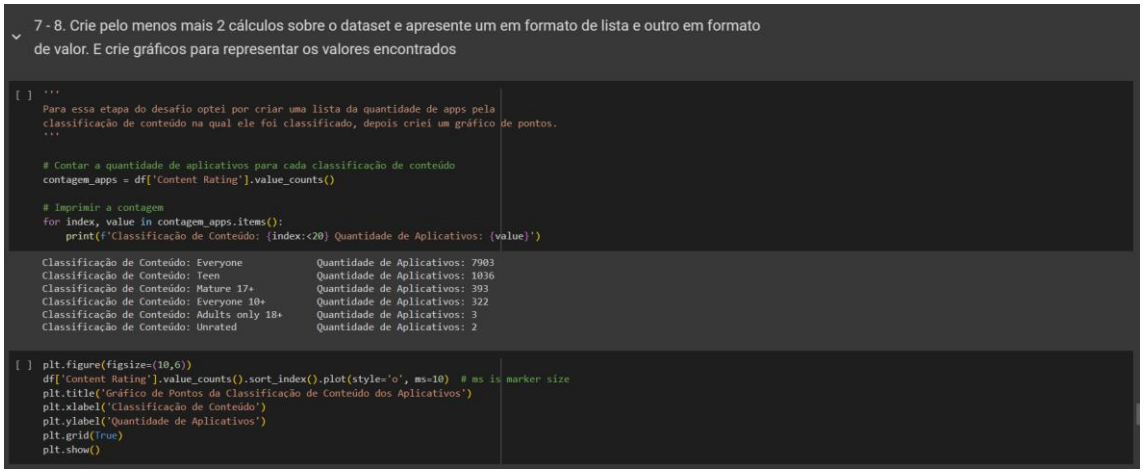
Figura 20 Top 10 reviews



11 – Exercício 7 e 8

Aqui optei por fazer o exercício 7 e 8 ao mesmo tempo. Primeiro fiz o cálculo com uma lista dos apps por classificação e ordenei de forma decrescente. Posteriormente, fiz um gráfico de pontos que eu não havia feito ainda. Por fim fiz um cálculo da quantidade de instalação por mês de 2018 e retornei o aplicativo com maior número de instalação e o mês. Como não consegui fazer o gráfico que eu queria, optei por retornar um gráfico mostrando a quantidade de instalação por cada mês de 2018.

Figura 21 Lista com apps mais instalados e gráfico de pontos



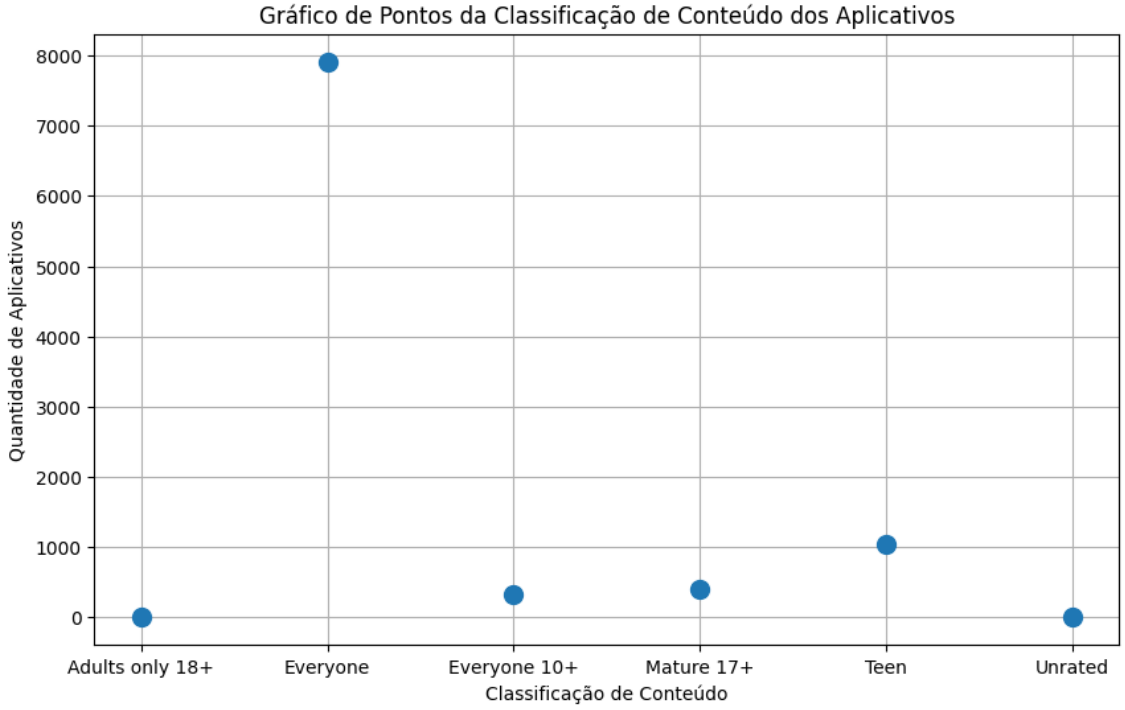


Figura 22 Quantidade de App instalados em 2018

```
[ ] from datetime import datetime

    # Criando uma nova coluna para o ano de atualização
    df['Update Year'] = df['Last Updated'].dt.year

    # Filtrando os dados para o ano de 2018
    df_2018 = df[df['Update Year'] == 2018]

    # Contando a quantidade de aplicativos instalados em 2018
    contagem_apps_2018 = df_2018['App'].count()

    print(f'Quantidade de aplicativos instalados no ano de 2018: {contagem_apps_2018}')

    Quantidade de aplicativos instalados no ano de 2018: 6287
```

Figura 23 Aplicativo mais instalado em 2018

```
    '''
    Queria nesse gráfico mostrar como um painel os valores encontrados nessa busca, mas com o matplotlib
    não consegui fazer. Então para esse último desafio optei por imprimir um gráfico com a quantidade de
    instalação por mês de 2018
    '''

    # Criando uma nova coluna para o ano e mês de atualização
    df['Update Year'] = df['Last Updated'].dt.year
    df['Update Month'] = df['Last Updated'].dt.month

    # Filtrando os dados para o ano de 2018
    df_2018 = df[df['Update Year'] == 2018]

    # Agrupando por aplicativo e mês de atualização e somando as instalações
    grouped_2018 = df_2018.groupby(['App', 'Update Month'])['Installs'].sum().reset_index()

    # Encontrando a linha com o maior número de instalações
    max_installs_row = grouped_2018.loc[grouped_2018['Installs'].idxmax()]

    print(f"O aplicativo com o maior número de instalações é: {max_installs_row['App']}")
    print(f"O maior número de instalações é: {max_installs_row['Installs']}")
    print(f"O mês em que ele teve o maior número de instalações é: {max_installs_row['Update Month']}")

    O aplicativo com o maior número de instalações é: Facebook
    O maior número de instalações é: 1000000000
    O mês em que ele teve o maior número de instalações é: 8
```

Figura 24 Gráfico com os aplicativos mais instalados por mês em 2018

```
    from datetime import datetime

    # Criando uma nova coluna para o ano de atualização
    df['Update Year'] = df['Last Updated'].dt.year

    # Filtrando os dados para o ano de 2018
    df_2018 = df[df['Update Year'] == 2018]

    # Agrupando por mês de atualização e somando as instalações
    grouped_2018 = df_2018.groupby(df_2018['Last Updated'].dt.month)['Installs'].sum().reset_index()

    # Plotando o resultado
    plt.figure(figsize=(10,6))
    plt.plot(grouped_2018['Last Updated'], grouped_2018['Installs'])
    plt.xlabel('Mês de Atualização')
    plt.ylabel('Quantidade de Instalações')
    plt.title('Quantidade de Instalações por Mês em 2018')
    plt.show()
```

